

Experiment No.	2
Experiment Title	Spam Classification using Naive Bayes and K-Nearest Neighbors (KNN) ¹
Student Name	Srinivasan M
Register Number	3122247001063
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	August 2, 2026

1 Objective

To implement and evaluate Naive Bayes (Gaussian, Multinomial, and Bernoulli variants) and K-Nearest Neighbors (KNN) classifiers on the Spambase dataset. The objective includes:

- Performing Exploratory Data Analysis (EDA) to understand feature distributions and class distributions.
- Rescaling features using MinMax normalization and preparing the dataset via stratified train-test splits.
- Training and evaluating the performance of Gaussian, Multinomial, and Bernoulli Naive Bayes models.
- Executing a parameter sweep over K-value in KNN, identifying the best k , and carrying out exhaustive hyperparameter tuning using Grid Search CV and Randomized Search CV.
- Investigating the impact of query algorithms (KD-Tree vs. Ball-Tree) on execution times.
- Benchmarking the classifiers across training/prediction times and cross-validation accuracies.

2 Problem Statement

Email spam poses significant communication security risks and inefficiencies. The goal is to build a binary classifier to accurately categorize a given email as “Spam” (Class 1) or “Ham” (Class 0).

- **Input:** A dataset comprising N emails, where each email is represented by $D = 57$ continuous attributes containing word frequency percentages, character counts, and capitalized word run-lengths.
- **Output:** A binary label $y \in \{0, 1\}$, where 0 indicates benign email (ham) and 1 indicates spam.
- **Objective:** Develop classifier models using Naive Bayes and KNN, optimize their hyperparameters, and evaluate them on metrics such as Accuracy, Precision, Recall, F1-Score, and ROC-AUC. Runtimes must also be compared to understand performance-latency trade-offs.

3 Dataset Description

The experiment is conducted on the UCI Spambase dataset. The structural parameters of the dataset are detailed below:

Dataset Name	Spambase Dataset
Dataset Source	UCI Machine Learning Repository
Number of Samples	4601
Number of Features	57 continuous attributes (+1 target class variable)
Number of Classes	2 (0: Ham, 1: Spam)
Missing Values	None (0 missing entries)
Train-Test Split	80% Train (3680 samples), 20% Test (921 samples) – Stratified

The features consist of:

- 48 features indicating the percentage of words in the email matching specific target words (e.g., “make”, “address”, “our”).
- 6 features indicating the percentage of characters matching specific glyphs (e.g., ‘\$’, ‘!’, ‘[’).
- 3 run-length features tracking uppercase letters (mean run length, maximum run length, and total run length).

4 Exploratory Data Analysis

To comprehend the characteristics of the dataset, several visualization techniques were implemented.

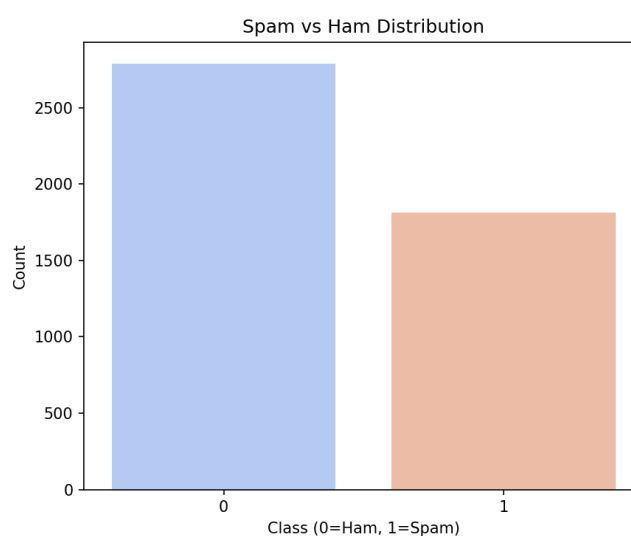


Figure 1: Spam vs Ham Class Distribution

Inference: The class distribution shows that 2,788 samples (60.6%) are ham and 1,813 samples (39.4%) are spam. Although a class imbalance exists, it is relatively moderate. It indicates that classification evaluation requires robust metrics like F1-Score and ROC-AUC, since accuracy alone can be misleading in extremely unbalanced contexts.

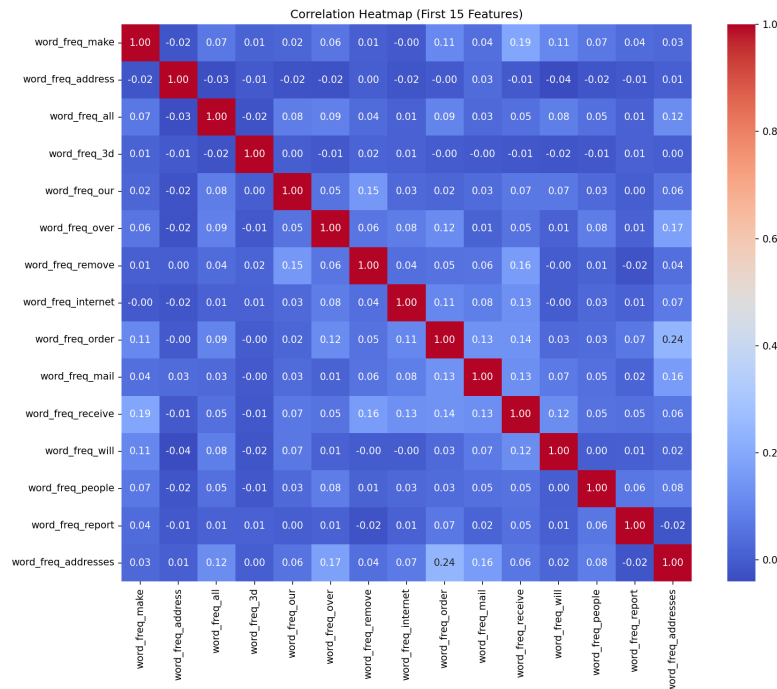


Figure 2: Correlation Heatmap of first 15 features

Inference: The heatmap of the correlation coefficients among the first 15 word-frequency features reveals that most attribute pairs share weak linear relationships, with coefficients close to zero. Some expected positive correlation is visible between logical word pairings. The general absence of multi-collinearity aligns well with the conditional independence assumption required for Naive Bayes models.

Inference: The histograms depict heavily right-skewed and zero-inflated distributions. For most word features, the percentage frequency is zero in the majority of emails. This sparsity indicates that classification techniques relying on distance metrics will be heavily skewed towards features with higher magnitudes and values, reinforcing the necessity of scale normalization.

Inference: The boxplots show a massive presence of outliers across all features, along with extreme differences in scale (some run-lengths are orders of magnitude larger than word frequencies). This visualization demonstrates that distance-based classifiers like KNN will suffer from scale bias if left unscaled, highlighting why feature standardization or normalization is required.

5 Data Preprocessing

Consistent preprocessing steps were executed:

1. **Missing Value Handling:** Checked using `isnull().sum()` showing zero null cells. No imputation was needed.

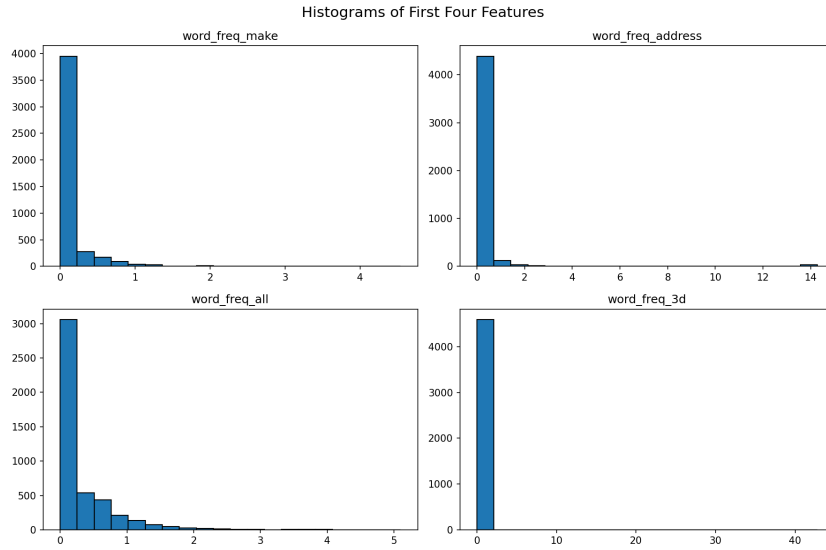


Figure 3: Frequency distribution histograms of first four features

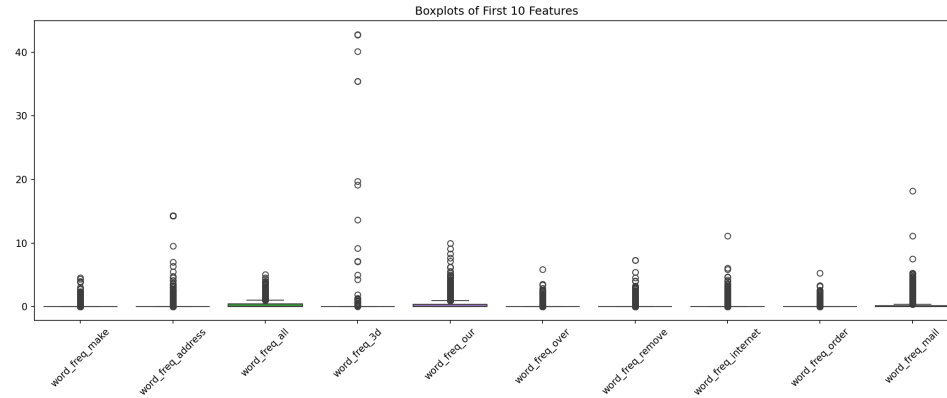


Figure 4: Boxplots of first 10 features

2. **Feature Scaling:** Since KNN determines nearest neighbors using distance metrics, the 57 independent variables were scaled into the range $[0, 1]$ using MinMax Normalization:

$$X_{scaled} = \frac{X - X_{min}}{X_{max} - X_{min}} \quad (1)$$

This scales uppercase count values and frequency percentages onto a shared interval, preventing features with large scales from dominating distance metrics.

3. **Train-Test Split:** An 80-20 stratified partition was applied, creating a training set of 3,680 instances and a test set of 921 instances. Stratification ensures that the 60.6 : 39.4 ratio of ham to spam classes is preserved strictly in both train and test partitions.

6 Machine Learning Algorithm

6.1 Naive Bayes Classifiers

Naive Bayes is a generative classification algorithm based on Bayes' Theorem with the assumption of conditional independence among features given the class label.

Mathematical Formulation

Bayes' Theorem calculates the posterior probability of a class $y \in \{0, 1\}$ given a feature vector $\mathbf{x} = [x_1, x_2, \dots, x_d]^T$:

$$P(y | \mathbf{x}) = \frac{P(\mathbf{x} | y)P(y)}{P(\mathbf{x})} \quad (2)$$

Assuming conditional independence of features ($P(x_i | y, x_j) = P(x_i | y)$):

$$P(y | \mathbf{x}) \propto P(y) \prod_{i=1}^d P(x_i | y) \quad (3)$$

The predicted class label is:

$$\hat{y} = \arg \max_{y \in \{0,1\}} P(y) \prod_{i=1}^d P(x_i | y) \quad (4)$$

Depending on the distribution model for likelihood $P(x_i | y)$, different sub-types are trained:

1. **Gaussian Naive Bayes (GNB)**: Assumes continuous features follow a normal distribution within each class:

$$P(x_i | y) = \frac{1}{\sqrt{2\pi\sigma_{y,i}^2}} \exp\left(-\frac{(x_i - \mu_{y,i})^2}{2\sigma_{y,i}^2}\right) \quad (5)$$

2. **Multinomial Naive Bayes (MNB)**: Typically models discrete term counts. For continuous MinMax normalized attributes (which act as pseudo-fractional counts), the likelihood is:

$$P(i | y) = \frac{N_{yi} + \alpha}{N_y + \alpha d} \quad (6)$$

where $\alpha \geq 0$ represents the Laplace smoothing parameter.

3. **Bernoulli Naive Bayes (BNB)**: Assumes ternary or binary variables. Independent features are binarized based on a user-defined threshold t :

$$P(x_i | y) = p_{y,i}^{B(x_i)} (1 - p_{y,i})^{1-B(x_i)} \quad \text{where } B(x_i) = \mathbb{I}(x_i > t) \quad (7)$$

6.2 K-Nearest Neighbors (KNN) Classifier

KNN is a non-parametric, instance-based supervised learning algorithm. Rather than constructing an explicit model during training, it saves historical test data and performs classification on Query points locally.

6.2.1 Working Principle

For any query sample $\mathbf{q}$, KNN identifies the k closest data objects in the normalized training set based on a distance metric, and outputs a consensus vote of their class tags.

- **Euclidean Distance (L_2 Norm):**

$$d_2(\mathbf{x}, \mathbf{q}) = \sqrt{\sum_{i=1}^d (x_i - q_i)^2} \quad (8)$$

- **Manhattan Distance (L_1 Norm):**

$$d_1(\mathbf{x}, \mathbf{q}) = \sum_{i=1}^d |x_i - q_i| \quad (9)$$

- **Weighting Schemes:**

- *Uniform*: Every neighbor's vote carries equal weight.
- *Distance*: Neighbor votes are weighted inversely to distance, $w = 1/d$, giving closer points larger influence.

6.2.2 Acceleration Trees

Brute-force lookup is $O(Nd)$, which is slow. Acceleration structures subdivide search regions:

1. **KD-Tree**: Space-partitioning tree that splits cells along axes progressively. Efficient for low to moderate dimensions ($d < 20$).
2. **Ball-Tree**: Groups points into nested hyper-spherical bounding areas. Better suited for high dimensions.

7 Experimental Setup

The experiments were conducted in the following hardware and software environment:

Component	Value
Python Version	3.14.2
Scikit-Learn Version	1.8.0
IDE	Jupyter Notebook / VS Code
Operating System	Windows 11
Hardware	Intel/AMD Multi-Core CPU, x86_64

Table 2: System Configurations

8 Hyperparameter Selection

Hyperparameter spaces configured for exploration are described below:

Hyperparameter Field	Tested Grid / Range Space
Standard KNN k Sweep	$k \in \{1, 3, 5, 7, 9, 11\}$
GridSearchCV Space	$k \in [1, 3, 5, 7, 9, 11]$, weights $\in [\text{uniform}, \text{distance}]$, metric $\in [\text{euclidean}, \text{manhattan}]$, alg $\in [\text{auto}, \text{kd_tree}, \text{ball_tree}]$
RandomizedSearchCV Space	$k \in [1, 19]$, weights $\in [\text{uniform}, \text{distance}]$, metric $\in [\text{euclidean}, \text{manhattan}]$, alg $\in [\text{auto}, \text{kd_tree}, \text{ball_tree}]$

Table 3: Hyperparameter Search Space

9 Results

Below is the compiled performance output summarizing baseline versions of the models:

Classifier Description	Accuracy	Precision	Recall	F1-score	ROC-AUC	Train Time (s)
Gaussian Naive Bayes	0.832790	0.714579	0.958678	0.818824	0.922917	0.003824
Multinomial Naive Bayes	0.893594	0.943144	0.776860	0.851964	0.962454	0.002870
Bernoulli Naive Bayes	0.634093	0.964286	0.074380	0.138107	0.587503	0.006753
Best KNN ($k = 5$)	0.902280	0.876033	0.876033	0.876033	0.941944	0.002015

Table 4: Classification Performance Summary

10 Result Visualization

10.1 Naive Bayes Performance Plots

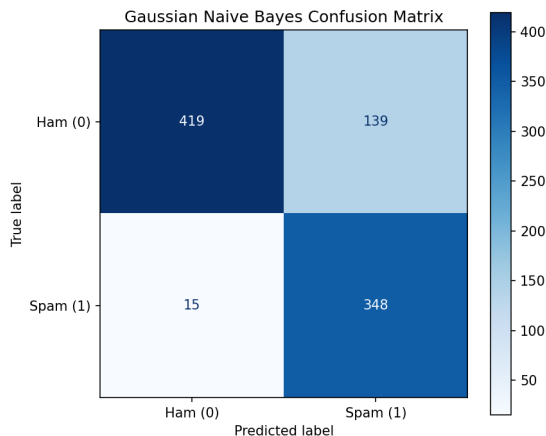


Figure 5: GNB Confusion Matrix

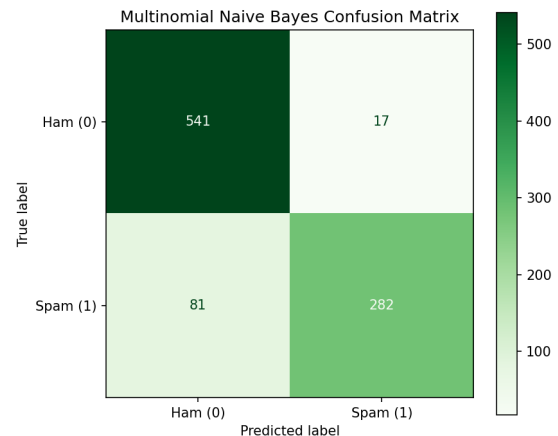


Figure 6: MNB Confusion Matrix

Inference: The Gaussian Naive Bayes model shows solid discriminative ability with an AUC of 0.9229. However, its confusion matrix reveals vertical imbalance; it identifies 348 out of 363 spam cases (Recall = 95.87%) but causes 139 false positives, drag down Precision to 71.46%. The Multinomial Naive Bayes model yields a superior AUC of 0.9625, and manages false positives much more effectively (only 17 false positives), resulting in a Precision of 94.31%.

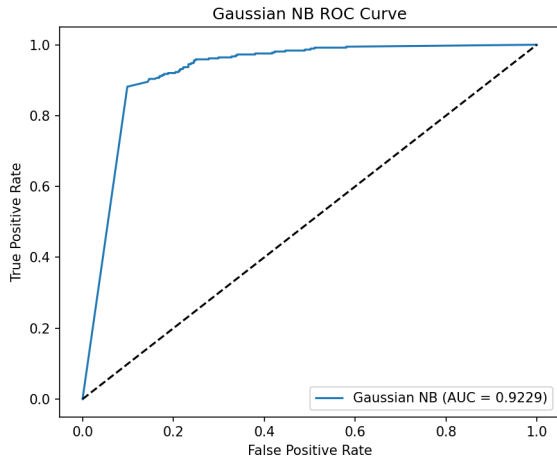


Figure 7: GNB ROC Curve

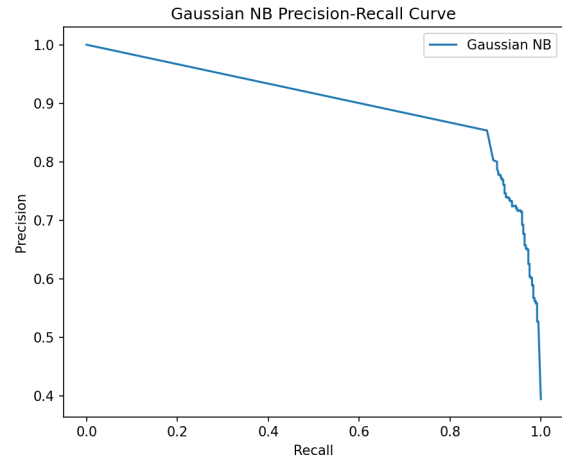


Figure 8: GNB Precision-Recall Curve

10.2 KNN Performance Plots

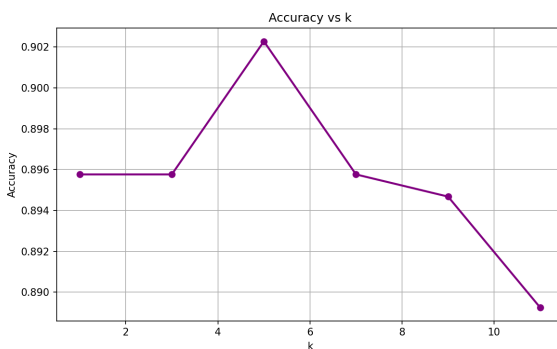
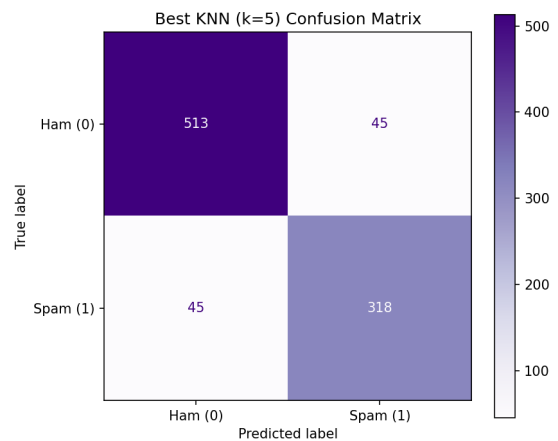
Figure 9: Accuracy vs. k values

Figure 10: Best KNN Confusion Matrix

Inference: The parameter sweep reveals that $k = 5$ achieves the highest baseline test accuracy of 90.23%. Lower k values ($k = 1, 3$) display minor overfitting, while higher k values ($k \geq 11$) experience underfitting as local decision boundaries soften. The confusion matrix for $k = 5$ displays balanced errors, registering 318 true spam cases and 513 true ham cases, culminating in symmetric precision and recall metrics of 87.60%.

10.3 Cross Validation and Execution Time Benchmarking

Inference: Cross-validation confirms that Best KNN is highly stable, achieving a mean accuracy of 89.87% compared to Gaussian NB's 81.55%. Evaluating runtimes highlights the core computational trade-offs: GNB and MNB train and predict near-instantaneously (~ 3 ms). Conversely, while KNN train phase is very fast (2.0 ms) because it is a lazy learner, its prediction phase is orders of magnitude slower (14.8 ms) due to the need to compute distances to all training samples.

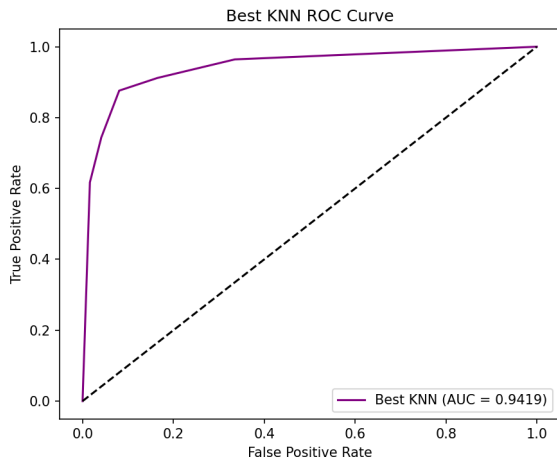


Figure 11: Best KNN ROC Curve

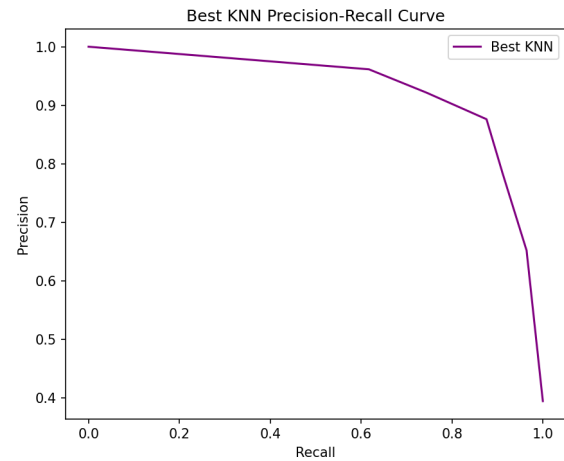


Figure 12: Best KNN PR Curve

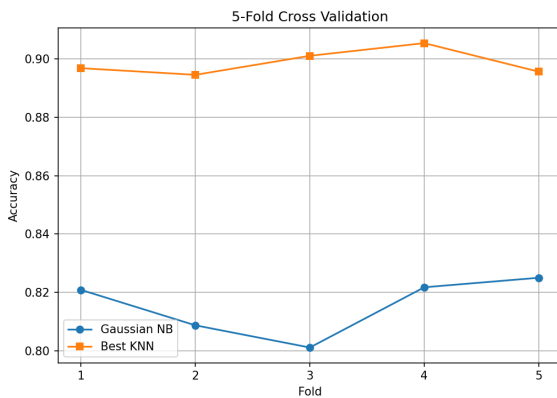


Figure 13: 5-Fold Cross Validation Scores

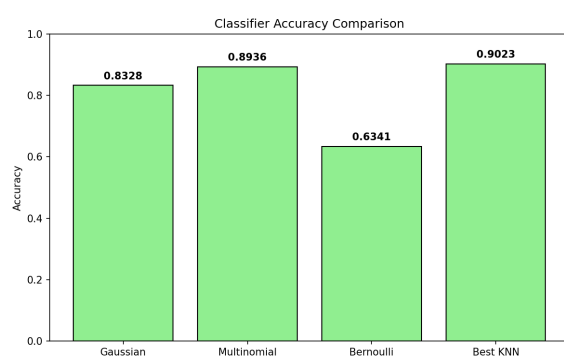


Figure 14: Classifier Comparison Bar Chart

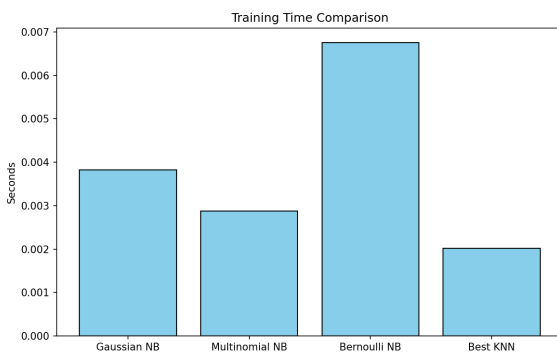


Figure 15: Training Time Runtimes

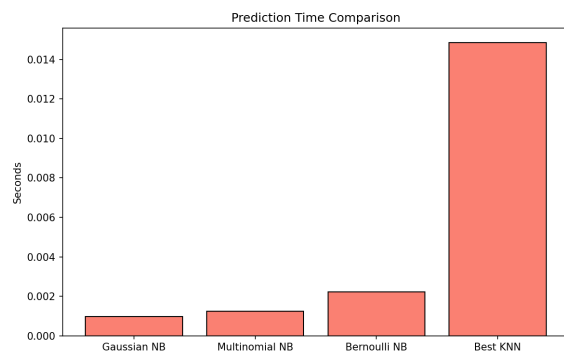


Figure 16: Prediction Time Runtimes

10.4 SearchCV Tuning Results

Inference: The GridSearchCV heatmap demonstrates that Manhattan distance consistently outperforms Euclidean distance across all tested k configurations. The grid search located the parameter set `{n_neighbors: 7, metric: 'manhattan', weights: 'distance'}` with a 5-fold CV score of 91.79% in 15.59 seconds. RandomizedSearchCV sampled 20 iterations, locating a configuration of `{n_neighbors: 12, metric: 'manhattan', weights:`

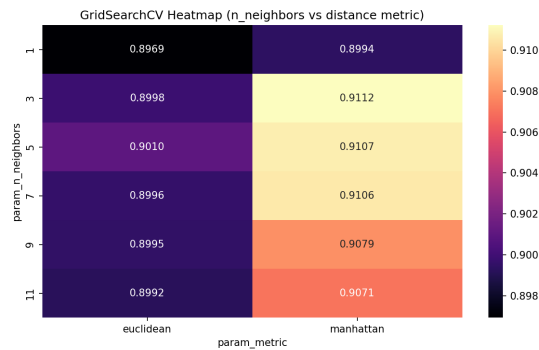


Figure 17: GridSearchCV Accuracy Heatmap

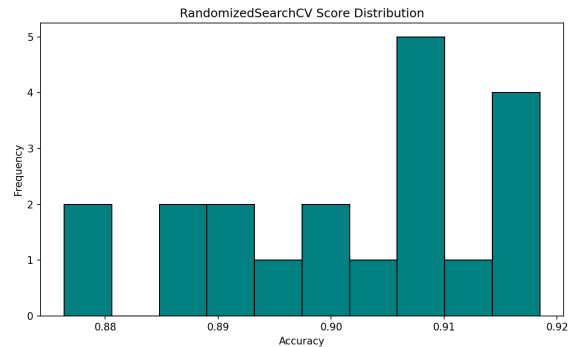


Figure 18: RandomizedSearchCV Scores

‘distance’} with a slightly higher CV score of 91.85% in only 3.07 seconds. This demonstrates that random search can be substantially faster ($5\times$) while yielding competitive or superior results compared to exhaustive search.

11 Discussion

A review of the experimental outputs reveals critical insights:

- **Bernoulli Naive Bayes Limitation:** BNB yielded a low spam detection accuracy of 63.41% and an extremely low Recall of 7.44%. This occurs because BNB binarizes values based on threshold $t = 0.5$. In the Spambase dataset, word frequency percentages are predominantly small fractions < 0.5 . This conversion to zero causes a severe loss of informational detail.
- **Gaussian Naive Bayes Performance:** GNB achieved a high Recall of 95.87% (only 15 missed spam instances). However, its assumption that features follow a normal distribution causes high false positives (139 ham emails flagged as spam), which decreases precision to 71.46%. This indicates that GNB is overly sensitive for spam filtering, where false positives (filtering legitimate emails) must be minimized.
- **Multinomial Naive Bayes Suitability:** MNB performed surprisingly well, achieving an accuracy of 89.36% and a precision of 94.31%. This demonstrates that tracking continuous word frequency magnitudes is highly effective for text filtering, keeping false positives low (1.85% rate) which is crucial for email classification in production.
- **KNN Acceleration Algorithms:** Table 5 shows empirical runtime testing of KD-Tree vs. Ball-Tree search for finding neighbors ($k = 5$): Both strategies resolve

Tree Strategy	Accuracy	Training Time (s)	Prediction Time (s)
KD-Tree	0.902280	0.021473	0.114488
Ball-Tree	0.902280	0.018124	0.122514

Table 5: KD-Tree vs. Ball-Tree Performance

the exact same set of nearest neighbors, leading to identical accuracies. KD-Tree

demonstrates slightly faster prediction lookup times (114.4 ms vs. 122.5 ms) for our 57-dimensional space.

12 Conclusion

This study evaluated Naive Bayes and KNN algorithms on the Spambase dataset.

- Naive Bayes models (especially Multinomial) are highly suitable for high-throughput spam filtering, achieving high accuracy (89.36%), exceptional precision (94.31%), and near-zero prediction latency (~ 1.2 ms).
- K-Nearest Neighbors yields superior overall predictive performance, with 5-fold cross-validated accuracy reaching 91.85% under optimal hyperparameters. However, this comes at the cost of high CPU prediction latency (~ 15 ms in lazy setup; ~ 114 ms when building recursive search trees).
- In a real-world email filtering system, Multinomial Naive Bayes is generally preferred due to its combined precision and low execution latency, which prevents legitimate emails from being incorrectly categorized as spam while minimizing prediction delays.

13 References

1. F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
2. M. Hopkins, E. Reeber, G. Forman, and J. Suermondt, “Spambase Dataset,” UCI Machine Learning Repository [Online]. Available: <https://archive.ics.uci.edu/ml/datasets/spambase>.
3. S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Englewood Cliffs, NJ, USA: Prentice Hall, 2020.